

# **INSTITUTO TECNOLÓGICO DE MAZATLÁN**

## **Ingeniería en Sistemas Computacionales**

**Materia:** Sistemas Operativos

**Docente:** Miguel Ángel Barrón Hernández

**Actividad:** Implementación de un Demonio Continuo con Supervisión en Systemd

**Alumno:** Israel Carmona

**Fecha:** 29 de Septiembre de 2026

## **1. INTRODUCCIÓN**

En la arquitectura de los sistemas operativos modernos de la familia Linux, existen servicios críticos que no dependen de sesiones de usuario interactivas ni de una planificación horaria estática. Estos procesos, denominados demonios (*daemons*), deben ejecutarse de manera continua e ininterrumpida en segundo plano para supervisar recursos vitales. El presente reporte documenta el diseño, despliegue y análisis de un demonio de monitoreo continuo gestionado mediante systemd, evaluando su resistencia y capacidad de autorrecuperación frente a señales de terminación forzada del kernel.

## **2. JUSTIFICACIÓN TEÓRICA DEL SISTEMA OPERATIVO**

### **2.1. Arquitectura de Demonios y Desacoplamiento de la Terminal**

Tradicionalmente, un proceso ejecutado desde una terminal está asociado a la sesión del usuario. Si la terminal se cierra, el kernel envía una señal SIGHUP provocando su terminación. Un demonio formal rompe este acoplamiento operando en un ciclo infinito en segundo plano sin interfaz de control directa, redirigiendo su salida hacia archivos de bitácora locales.

### **2.2. Semántica de Señales POSIX: La Señal SIGKILL (kill -9)**

El estándar POSIX define señales para el control de procesos. La señal SIGKILL (identificador numérico 9) es destructiva e ineludible. A diferencia de SIGTERM, SIGKILL no puede ser capturada ni ignorada por el proceso. Al recibirla, el kernel de Linux detiene inmediatamente la ejecución, destruye el espacio de memoria del proceso y purga su Bloque de Control de Proceso (PCB). Por tanto, la supervivencia del servicio recae completamente en un supervisor externo.

### **2.3. Systemd como Supervisor de Ciclo de Vida (PID 1)**

systemd es el primer proceso del espacio de usuario (PID 1). A través de su unidad de servicio (.service), se emplearon las directivas Restart=always y RestartSec=2. Al morir

el demonio repentinamente por el kill -9, el kernel notifica a systemd. Al detectar la terminación anómala, el supervisor espera 2 segundos y ejecuta una nueva bifurcación (fork), resucitando el servicio con un nuevo PID en menos de 5 segundos.

### 3. ANÁLISIS REFLEXIVO DE LAS OBSERVACIONES PRÁCTICAS

Durante la experimentación práctica en la terminal de Ubuntu se observó lo siguiente:

1. **Persistencia de la Bitácora:** El archivo demonio\_salud.log evidenció registros periódicos consecutivos capturando fecha con segundos, PID activo y RAM consumida, demostrando que el proceso mantuvo una ejecución continua no bloqueante.
2. **Resiliencia ante kill -9:** Al ejecutar la terminación forzada, el proceso cesó. Sin embargo, al consultar el estado con systemctl apenas tres segundos después, el servicio ya se encontraba nuevamente en estado active (running).
3. **Asignación Dinámica de PID:** El registro en la bitácora comprobó que el PID original fue liquidado definitivamente por el kernel. El nuevo proceso levantado por systemd adquirió un identificador numérico completamente distinto, reanudando la tarea de monitoreo de forma transparente para el sistema.

### 4. CONCLUSIÓN

La combinación de scripts y la administración mediante systemd constituye la arquitectura estándar para servicios desatendidos en Linux. Se demostró que la resiliencia operativa ante fallos críticos o señales ineludibles como kill -9 no se programa dentro del proceso mismo, sino delegando el control de ciclo de vida al gestor de inicio del sistema operativo.